

ORF307

Network optimization

Ioannis Akrotirianakis

Today's topics

- Network flows
- Minimum cost network flow problem
- Network flow solutions
- Examples: maximum flow, shortest path, assignment problem

- **Recap**

Primal and Dual basic feasible solutions

Optimality conditions of the **standard form** LP

Primal:

$$\begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & Ax = b \\ & x \geq 0 \end{aligned}$$

Dual:

$$\begin{aligned} \max \quad & -b^T y \\ \text{s.t.} \quad & A^T y + c \geq 0 \end{aligned}$$

Primal feasibility: $Ax = b, x \geq 0 \Rightarrow x_B = A_B^{-1}b \geq 0, x_N = 0 \Rightarrow x \geq 0$

Dual feasibility: $A^T y + c = 0$. Set $y = -A_B^{-T}c_B$. If $\bar{c} = c + A^T y \geq 0$ then y is dual feasible

(by construction) (reduced cost)

Let's see if $x_B = A_B^{-1}b$ and $y = -A_B^{-T}c_B$ are **optimal**?

Duality gap is zero: $c^T x + b^T y = c_B^T x_B + b^T (-A_B^{-T}c_B) = c_B^T x_B - c_B^T A_B^{-1}b = c_B^T x_B - c_B^T x_B = 0$

(by construction)

Optimal value function

$$p^*(u) = \min\{c^T x \mid Ax = b + u, \quad x \geq 0\}$$

Assumption: $p^*(0)$ is finite

Properties:

- $p^*(u) > -\infty$ everywhere (from the global lower bound)
- $p^*(u)$ is piecewise-linear on its domain

Local sensitivity analysis

u is now in a neighborhood of the origin

Original LP:

$$\begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & Ax = b \\ & x \geq 0 \end{aligned}$$

Optimal solution:

- Primal: $x_B^* = A_B^{-1}b$ and $x_i^* = -0$, $i \notin B$
- Dual: $y^* = -A_B^{-T}c_B$

Modified LP

$$\begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & Ax = b + u \\ & x \geq 0 \end{aligned}$$

Modified Dual

$$\begin{aligned} \max \quad & -(b + u)^T y \\ \text{s.t.} \quad & A^T y + c \geq 0 \end{aligned}$$

Modified optimal solution:

$$\begin{aligned} x_B^*(u) &= A_B^{-1}(b + u) = x_B^* + A_B^{-1}u \\ y^*(u) &= y^* \end{aligned}$$

The optimal basis does not change

Derivative of the optimal value function

Modified optimal solution:

$$x_B^*(u) = A_B^{-1}(b + u) = x_B^* + A_B^{-1}u$$

$$y^*(u) = y^*$$

Optimal value function:

$$\begin{aligned} p^*(u) &= c^T x^*(u) = c^T x + c_B^T A_B^{-1}u \\ &= p^*(0) - y^{*T}u \quad \rightarrow \quad (\textbf{affine for small } u) \end{aligned}$$

Local derivative:

$$\nabla p^*(u) = -y^* \quad \rightarrow \quad (y^* \text{ are called } \textbf{shadow prices})$$

-
- **Network flows**

Networks

Networks are prevalent in every aspect of our lives

Examples:

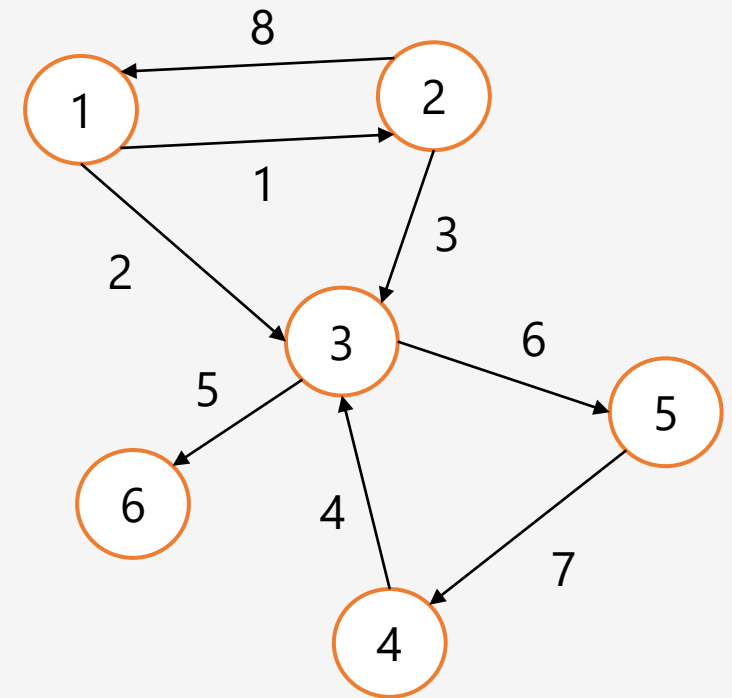
- Electrical and power networks
- Traffic networks
- Supply chain and logistics networks
- Social networks and recommendation systems
- Airline route scheduling
- Telecommunication and data networks
- Blood flow modeling
- Crime and fraud detection
- ...



Network modeling

Definition: A **network** (or a directed graph, or digraph) is a **set of m nodes and n directed arcs**.

- Arcs are ordered pairs of nodes (i, j) , where i is the **outgoing node** and j the **incoming node**.
- **Assumptions:**
 - There is **at most one arc** from node i to node j .
 - There are **no self-loops** (i.e., arcs from i to i)

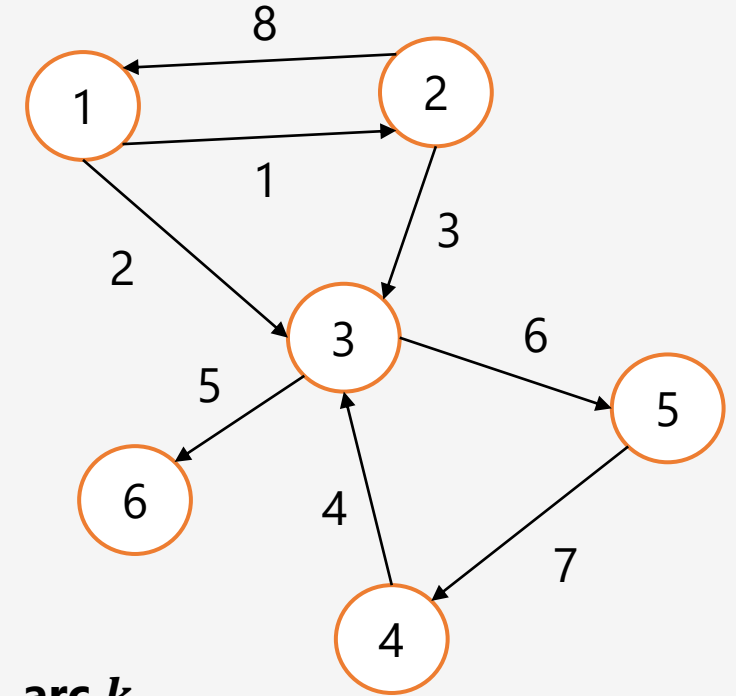


Ark-node incidence matrix

Definition: a matrix $A \in R^{m \times n}$ is called the **incidence matrix** of a network if

$$A_{ik} = \begin{cases} 1, & \text{If arc } k \text{ starts at node } i \\ -1, & \text{If arc } k \text{ ends at node } i \\ 0, & \text{otherwise} \end{cases}$$

Note: each column has one -1 and one 1



$$A = \begin{matrix} & \text{arc } k \\ \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & -1 \\ -1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & -1 & -1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \end{bmatrix} & \text{node } i \end{matrix}$$

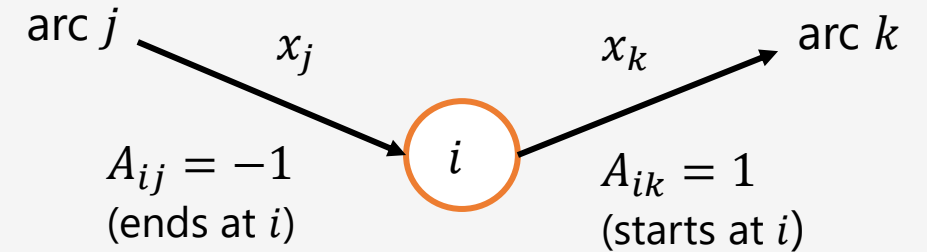
Network flow

Flow vector $x \in R^n$

- Its j -th element x_j represent the **amount that flows** through arc j
- x_j may represent: material, traffic, information, electricity, etc.

Total flow leaving node i

$$\sum_{k=1}^n A_{ik} x_k = (Ax)_i$$



External supply

Supply vector $b \in R^m$:

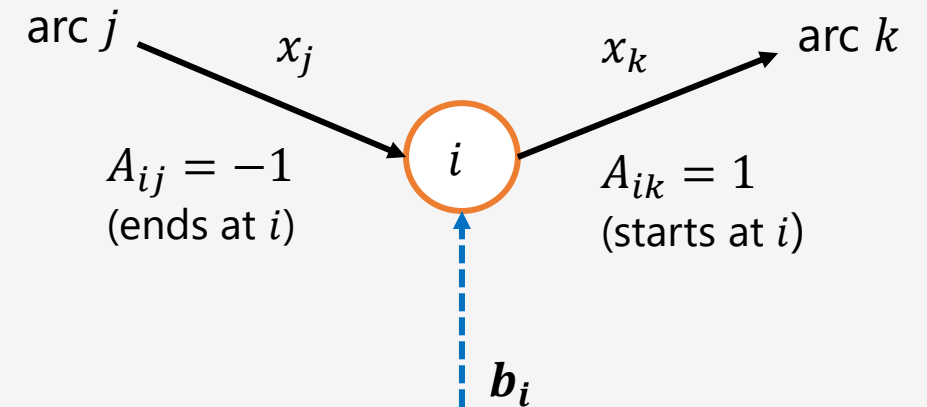
- b_i is the **external supply** at node i
 - If $b_i < 0$, it represents **demand** at node i
- We must have $\mathbf{1}^T \mathbf{b} = 0$
 - Total supply = Total demand

Balance equations

$$\sum_{j=1}^n A_{ij} x_j = (Ax)_i = b_i, \forall i \Rightarrow Ax = b$$

Total outgoing
flow at node i

Supply at node i



-
- **Minimum cost network flow**

Minimum cost network flow problem

General formulation

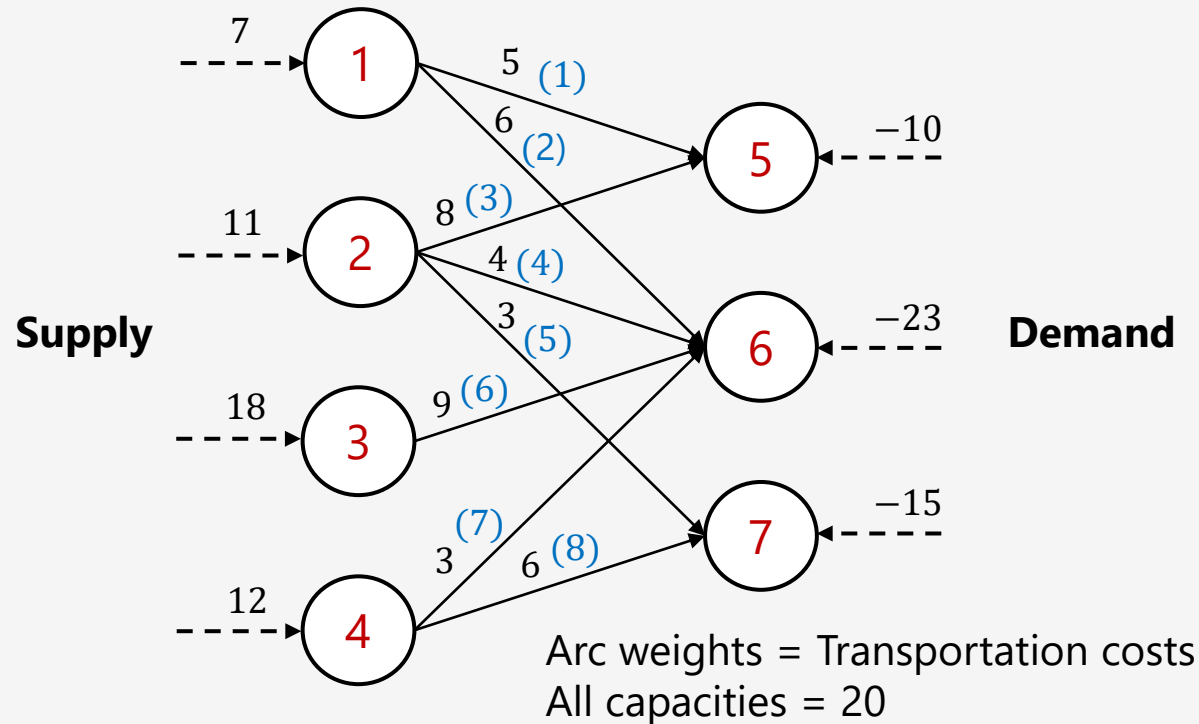
$$\begin{array}{ll} \min c^T x & c, x, u \in R^n \\ \text{s.t. } Ax = b & b \in R^m \\ & 0 \leq x \leq u \quad A \in R^{m \times n} \end{array}$$

- $c_i \rightarrow$ the **unit cost of flow** through arc i
- $x_i \rightarrow$ flow on arc i must be **nonnegative**
- $u_i \rightarrow$ the **maximum flow capacity** of arc i

Many network optimization problems are a special case of this model

Example - Transportation

Goal: compute the **optimal amount** $x \in R^n$ to be **transported** through the network so that the **demand is satisfied**



Transportation cost: $c = (5, 6, 8, 4, 3, 9, 3, 6)^T$

Supply/Demand: $sb = (7, 11, 18, 12, -10, -23, -15)^T$

Max. arc capacity: $u = (20, 20, 20, 20, 20, 20, 20, 20)^T$

$$A = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \end{matrix} & \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ -1 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & -1 & 0 & -1 & -1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 0 & -1 \end{bmatrix} \end{matrix}$$

Minimum cost network flow:

$$\begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & Ax = b \\ & 0 \leq x \leq u \end{aligned}$$

-
- **Network flow solutions**

Remove arc capacities

Goal: create equivalent network **without arc capacities**

Minimum cost network flow:

$$\min c^T x$$

$$s.t. \ Ax = b$$

$$0 \leq x \leq u$$



Minimum cost network flow (standard form LP):

$$\min \tilde{c}^T x$$

$$s.t. \ \tilde{A}\tilde{x} = \tilde{b}$$

$$\tilde{x} \geq 0$$

Remove arc capacities

Idea: use slack variables

$$x_j \leq u_j \Rightarrow x_j + s_j = u_j, s_j \geq 0$$

But the network structure is **lost**
(no longer a "+1" and a "-1" per column)

Network structure **recovered**
(new node and new arc)

$$\dots + x_j \dots = b_p$$

$$\dots - x_j \dots = b_q$$

$$x_j + s_j \dots = u_j \Rightarrow$$

solve for x_j

$$\Rightarrow x_j \dots = u_j - s_j$$

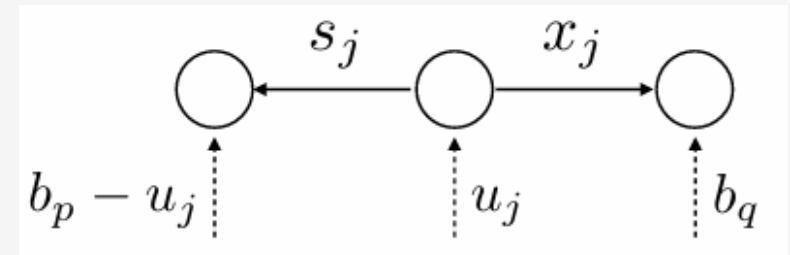
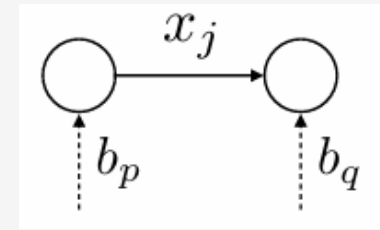
**Substitute to
 p -th constraint**

$$\dots - s_j = b_p - u_j$$

$$\dots - x_j + \dots = b_q$$

$$x_j \dots + s_j = u_j$$

**Nodes/arcs
interpretation**



Equivalent *un-capacitated* network

Minimum cost network flow (standard form LP):

$$\min \tilde{c}^T x$$

$$s.t. \tilde{A}\tilde{x} = \tilde{b}$$

$$\tilde{x} \geq 0$$

- Matrix A is **still** an arc-node incidence matrix
- But can we say something about the **extreme points??**

Total unimodularity

- **Definition:** a **minor** of a matrix $A \in R^{m \times n}$ is the determinant of a square submatrix of A .
- **Definition:** A matrix $A \in R^{m \times n}$ is **totally unimodular** if **all its minors** are $-1, 0$, or 1 .
- **Example:** a **node-arc incidence matrix** of a directed graph is **totally unimodular**

$$A = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & -1 \\ -1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & -1 & -1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \end{bmatrix}$$

Properties:

- The **entries** A_{ij} of A have values: $-1, 0$, or 1 .
- The **inverse** of any nonsingular square **submatrix** of A has entries $-1, 0$, or 1 .

Integrality theorem

Theorem: Consider a polyhedron $P = \{x \in R^n \mid Ax = b, x \geq 0\}$, where $A \in R^{m \times n}$ is a **totally unimodular matrix** and $b \in R^m$ is an **integer vector**. Then **all extreme points of P are integer vectors**.

Proof: recall that all **extreme points** are **basic feasible solutions** with

$$x_B = A_B^{-1}b \text{ and } x_i = 0, \forall i \notin B.$$

Also, A_B^{-1} has integer elements because A is totally unimodular matrix

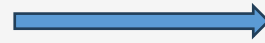
Note also that **b has integer elements**

Hence the vector $x_B = A_B^{-1}b$ must have integer elements, and so does x . ■

Implications for network and **combinatorial optimization**

Minimum cost network flow

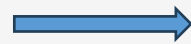
$$\begin{array}{ll}\min & c^T x \\ \text{s.t.} & Ax = b \\ & 0 \leq x \leq u\end{array}$$



If b and u are **integral vectors**, then the optimal solution x^* is **integral too**

Integer Linear Program (ILP)

$$\begin{array}{ll}\min & c^T x \\ \text{s.t.} & Ax = b \\ & 0 \leq x \leq u \\ & x \in \mathbb{Z}^n\end{array}$$



ILPs are very difficult to solve (*NP*-complete)

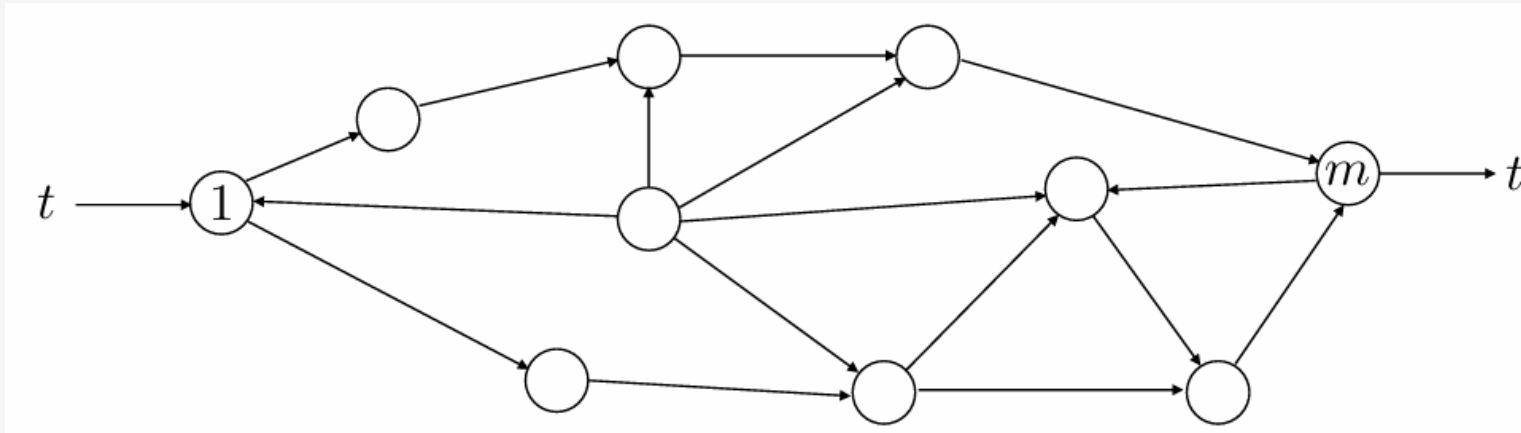


If A is **totally unimodular** and b, u are **integral**, then we can **relax** the **integrality constraint** $x \in \mathbb{Z}^n$, **solve the LP instead**, and **still get an integral solution**.

-
- **Examples**

Maximum flow problem

Goal: maximize the flow in the network from node 1 (**source**) to node m (**sink**)



$$\begin{aligned} \max \quad & t \\ \text{s.t.} \quad & Ax = te \\ & 0 \leq x \leq u \end{aligned} \quad e = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \\ -1 \end{bmatrix}$$

- Flow must conserve mass at each node

$$\text{FlowIN} = \text{FlowOUT}$$

- Flow must respect the capacities u of each arc

x : is the flow vector and t : is the demand at the sink node

Example:

- A city's **water utility** needs to **route water** from a main reservoir (**source**) to a distribution station (**sink**) through a **network of pipes**.
- Each pipe has a **maximum capacity**, and the **goal** is to **maximize the amount of water delivered to the station without exceeding these limits**.

Network structure:

- Node 1 is the source node
- Node 6 is the sink node (distribution node)
- Nodes 2,3,4,5 are junction nodes.

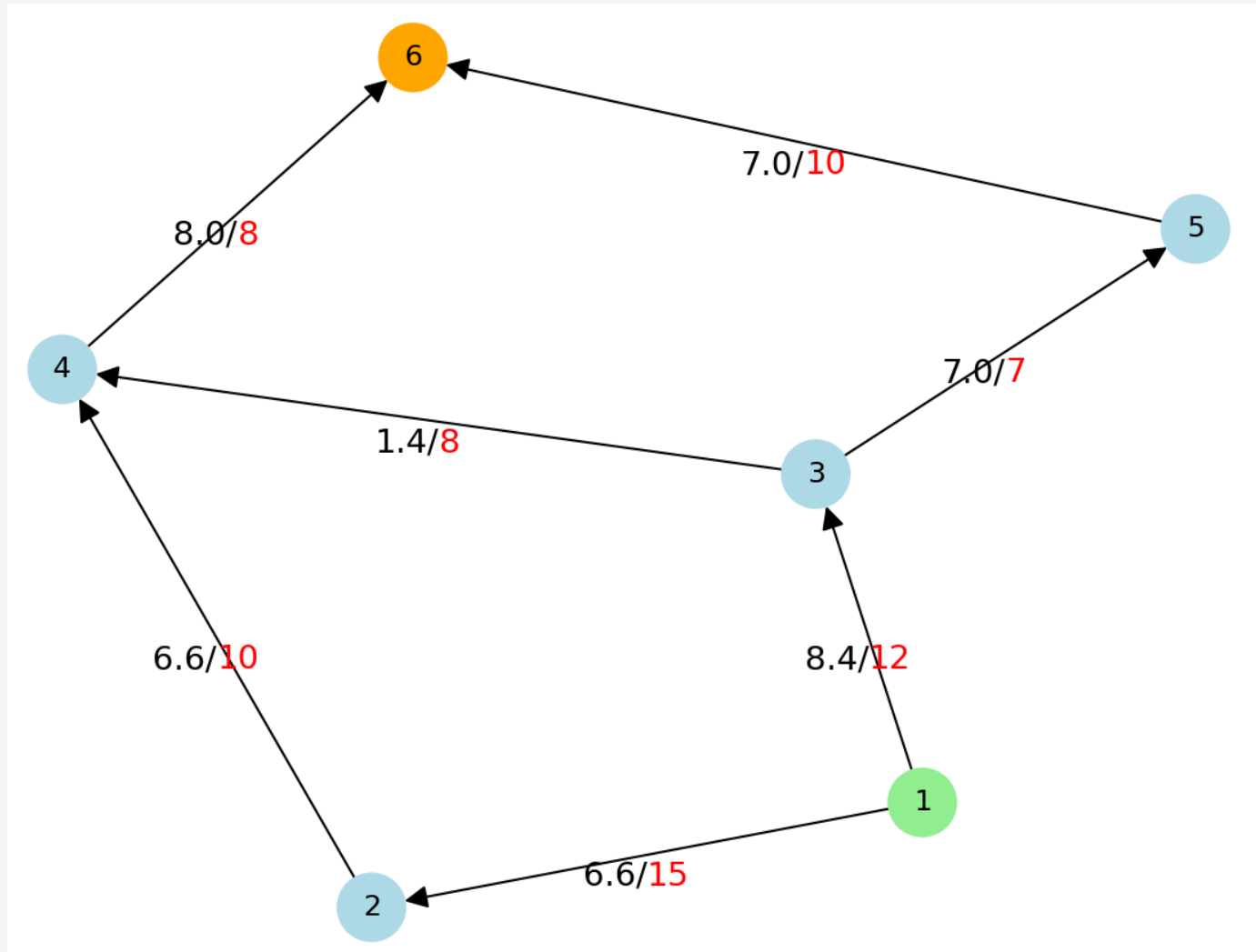
$$\max t$$

$$s.t. Ax = te$$

$$0 \leq x \leq u$$

$$A = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & -1 & -1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & -1 & -1 \end{bmatrix} \quad e = \begin{bmatrix} +1 \\ 0 \\ 0 \\ 0 \\ 0 \\ -1 \end{bmatrix} \quad u = \begin{bmatrix} 15 \\ 12 \\ 10 \\ 8 \\ 7 \\ 8 \\ 10 \end{bmatrix}$$

Example

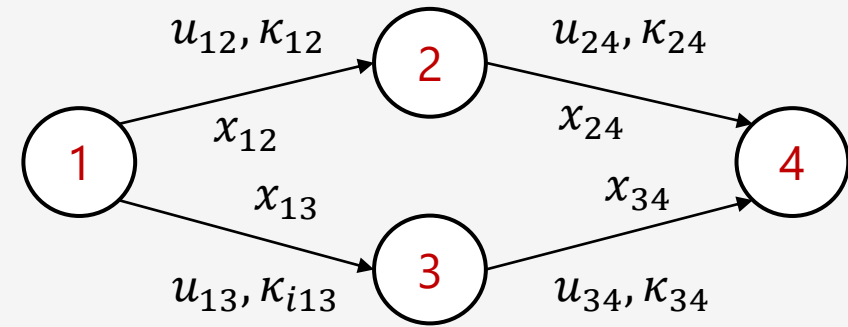


Capacities in red

Optimal flow in black

The Minimum Cost Flow (MCF) problem

- It is a **generalization** of the Max Flow Problem that introduces two new important concepts
 - κ_{ij} : the **cost per unit of flow** on each edge (i, j)
 - Each node i has a required **flow balance** b_i
 - Supply** node: $b_i > 0$ (source)
 - Demand** node: $b_i < 0$ (sink)
 - Transshipment** node: $b_i = 0$
- Network is represented as a **graph**: $G = (V, E)$
- Incoming nodes** to node j : $I(j) = \{i \in V \mid (i, j) \in E\}$
- Outgoing nodes** to node j : $O(j) = \{\ell \in V \mid (j, \ell) \in E\}$



$$\begin{aligned} \min \quad & \sum_{(i,j) \in E} \kappa_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{\ell \in O(j)} x_{j\ell} - \sum_{i \in I(j)} x_{ij} = b_j \quad \forall j \in V \\ & 0 \leq x_{ij} \leq u_{ij} \quad \forall (i, j) \in E \end{aligned}$$

$$\begin{aligned} \min \quad & \sum_{(i,j) \in E} \kappa_{ij} x_{ij} \\ \text{s.t.} \quad & Ax = b \\ & 0 \leq x \leq u \end{aligned} \quad b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_{|V|-1} \\ b_{|V|} \end{bmatrix}$$

Max Flow Problem as a Min Cost Flow problem

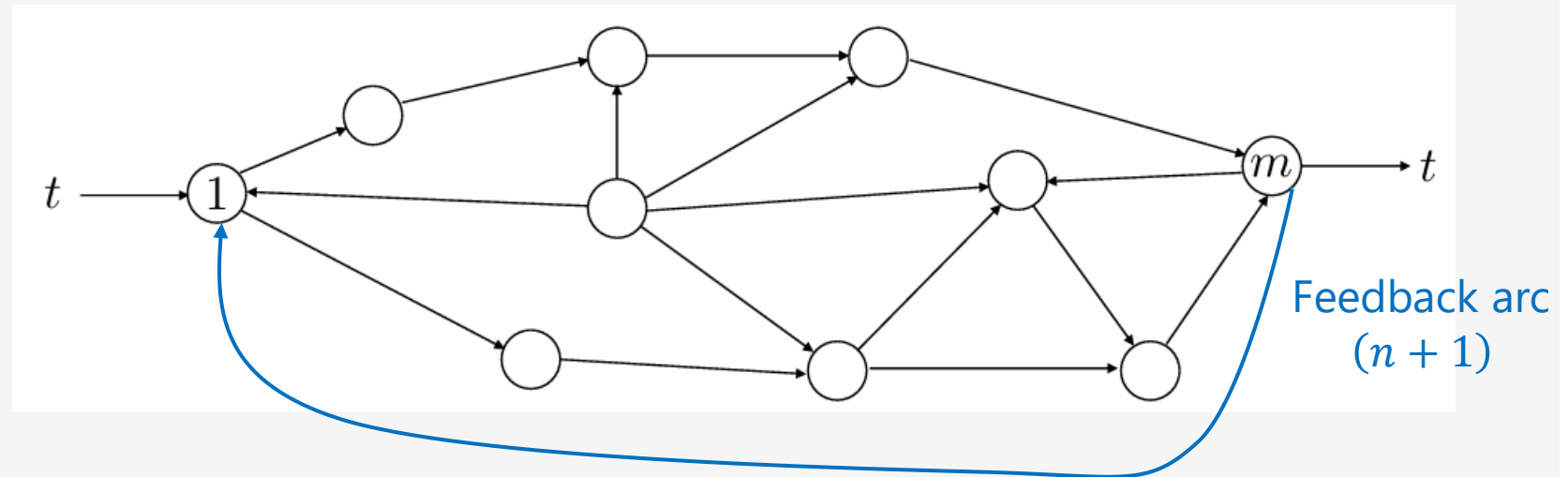
The steps to follow:

- Add a **feedback edge** to the Max Flow problem: from the sink node to the source node.
- Set **all costs** per unit flows to **zero**: $\kappa_{ij} = 0, \forall (i, j) \in E$
- Set the **cost from the sink node to the source node** to -1 : $\kappa_{m1} = -1$
- Set the **capacity from the sink node to the source node** to ∞ : $c_{m1} = \infty$
- Set **balances** to all nodes to **zero** : $b_i = 0$ ($FlowIN = FlowOUT$)
- Define the **objective**: **minimize** the negative flow from sink to source: $\min -x_{m1}$

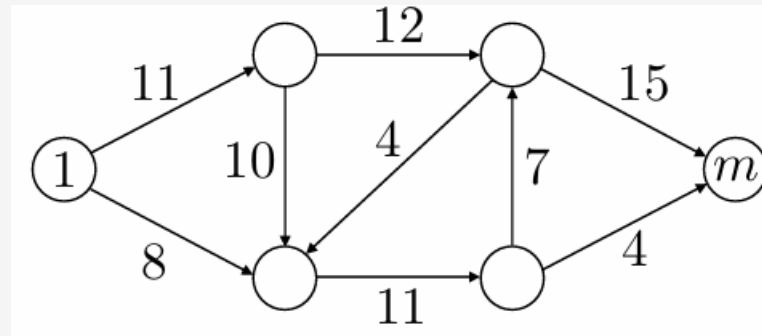
$$\min -t$$

$$s.t. \begin{bmatrix} A & -e \end{bmatrix} \begin{bmatrix} x \\ t \end{bmatrix} = 0$$

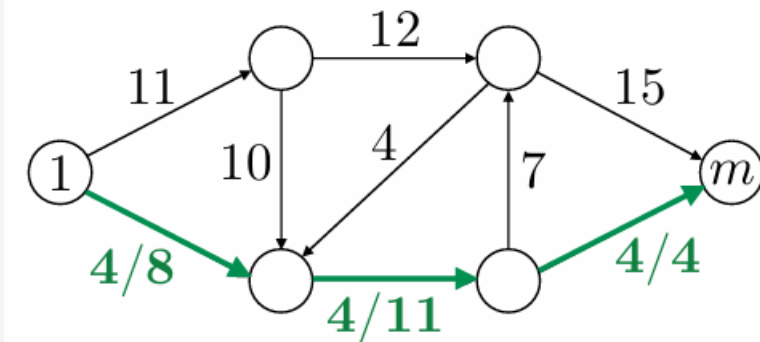
$$0 \leq \begin{bmatrix} x \\ t \end{bmatrix} \leq \begin{bmatrix} u \\ \infty \end{bmatrix}$$



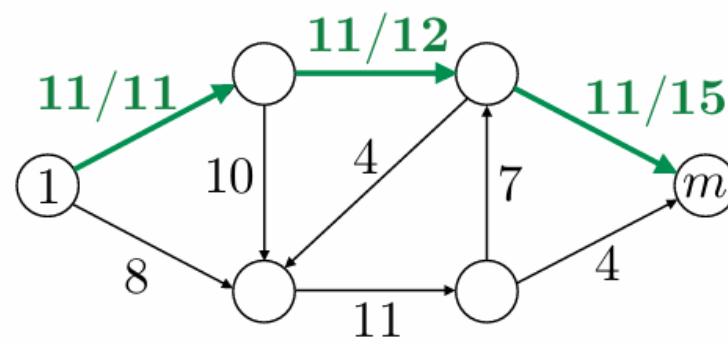
Maximum flow example – *different paths*



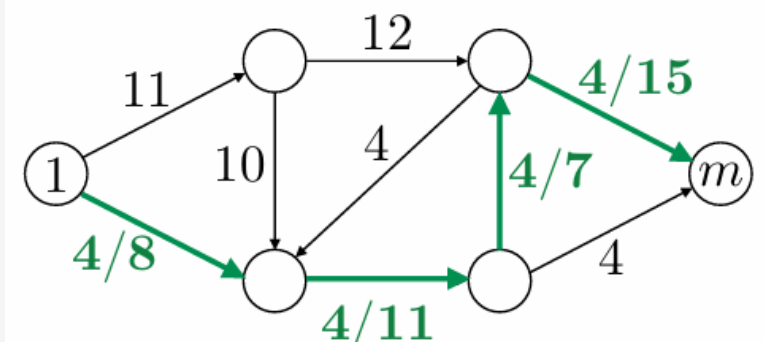
First flow



Second flow

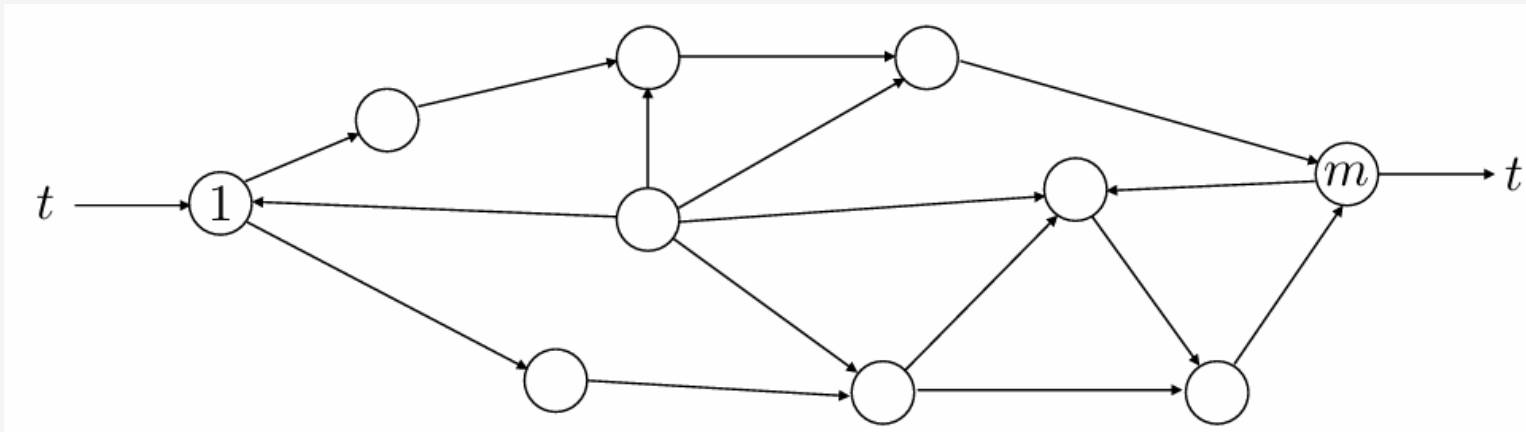


Third flow



Shortest path problem

Goal: find the shortest path between nodes 1 and m



Paths can be represented as **0 – 1 vectors**, $x \in \{0, 1\}^n$

$$\min c^T x$$

$$s.t. Ax = e$$

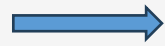
$$x \in \{0, 1\}^n$$

- c_j is the "length" of arc j
- $e = (1, 0, \dots, 0, -1)^T$
- The variables x_j can only take **0 or 1 values**
 - $x_j = 1$, if arc j is **in** the shortest path
 - $x_j = 0$, otherwise

Shortest path as minimum cost flow

(IP)

$$\begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & Ax = e \\ & x \in \{0,1\}^n \end{aligned}$$



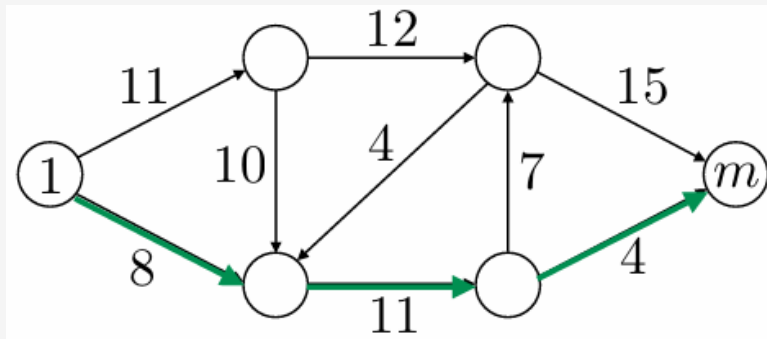
$$\begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & Ax = e \\ & 0 \leq x \leq 1 \end{aligned}$$

(LP-relaxation)
Easier to solve!



Extreme points satisfy $x_i \in \{0, 1\}$

Example (arc costs shown)



$$c = (11, 8, 10, 12, 4, 11, 7, 15, 4)^T$$

$$x^* = (0, 1, 0, 0, 0, 1, 0, 0, 1)^T$$

$$c^T x^* = 24$$

Assignment problem

Goal: match N people to N tasks

- Each **person** assigned to one **task**
- Each **task** assigned to one **person**
- C_{ij} : the **cost of matching** person i to task j

LP formulation:

$$\min \sum_{i=1}^N \sum_{j=1}^N C_{ij} X_{ij}$$

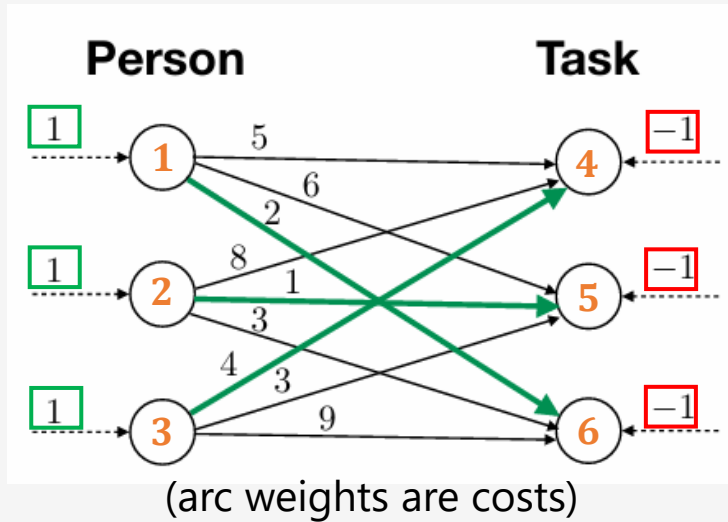
$$\text{s.t. } \sum_{i=1}^N X_{ij} = 1, j = 1, \dots, N$$

$$\sum_{j=1}^N X_{ij} = 1, i = 1, \dots, N$$

$$X_{ij} \in \{0,1\}, \forall i, j$$

How can we define the network?

Task assignments as minimum cost network flow



Minimum cost network flow

$$\min c^T x$$

$$s.t. Ax = b$$

$$0 \leq x \leq 1$$

Extreme points satisfy $x_i \in \{0,1\}$

$$A = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ -1 & 0 & 0 & -1 & 0 & 0 & -1 & 0 & 0 \\ 0 & -1 & 0 & 0 & -1 & 0 & 0 & -1 & 0 \\ 0 & 0 & -1 & 0 & 0 & -1 & 0 & 0 & -1 \end{bmatrix}$$

$$c = (5,6,2,8,1,3,4,3,9)^T, \quad b = (1,1,1,-1,-1,-1)^T$$

Optimal solution:

$$x^* = (0,0,1,0,1,0,0,0,1)^T$$

$$c^T x^* = 7$$

References

- D. Bertsimas & J. Tsitsiklis: Introduction to Linear Optimization
 - Ch. 7: Network flow problems
- R. Vanderbei: Linear Programming: Foundations and Extensions
 - Ch. 14: Network flow problems
 - Ch. 15: Applications

Next lecture

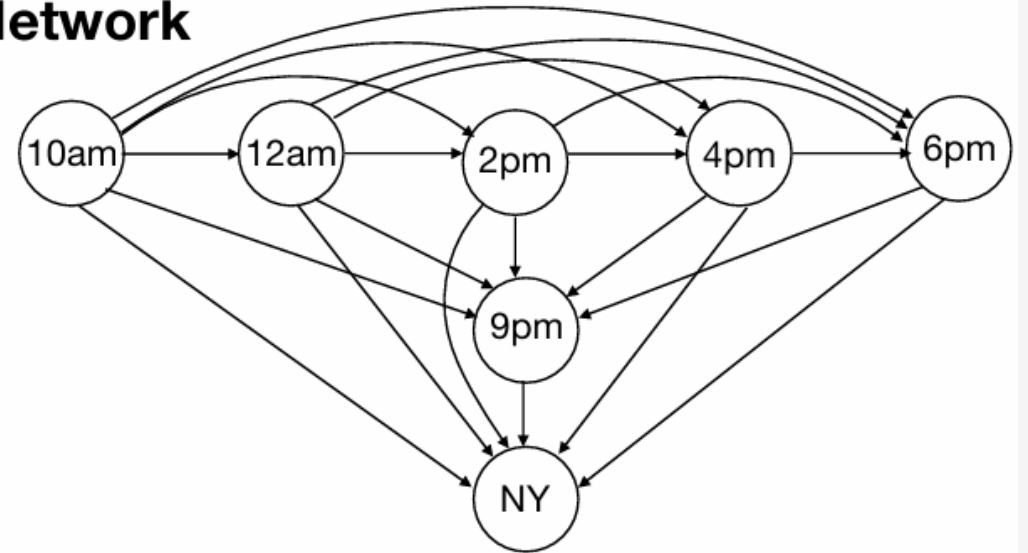
- Interior Point Algorithms

- **Appendix**

Example – Airline passenger routing

- United Airlines has 5 flights per day from Boston to New York
- Flight capacities: $u = (100, 100, 100, 150, 150)^T$
- Costs: \$50/hour of delay
- Last option: 9pm flight with other company (pay additional cost of \$75)
- Today's reservations: $(110, 118, 103, 161, 140)^T$

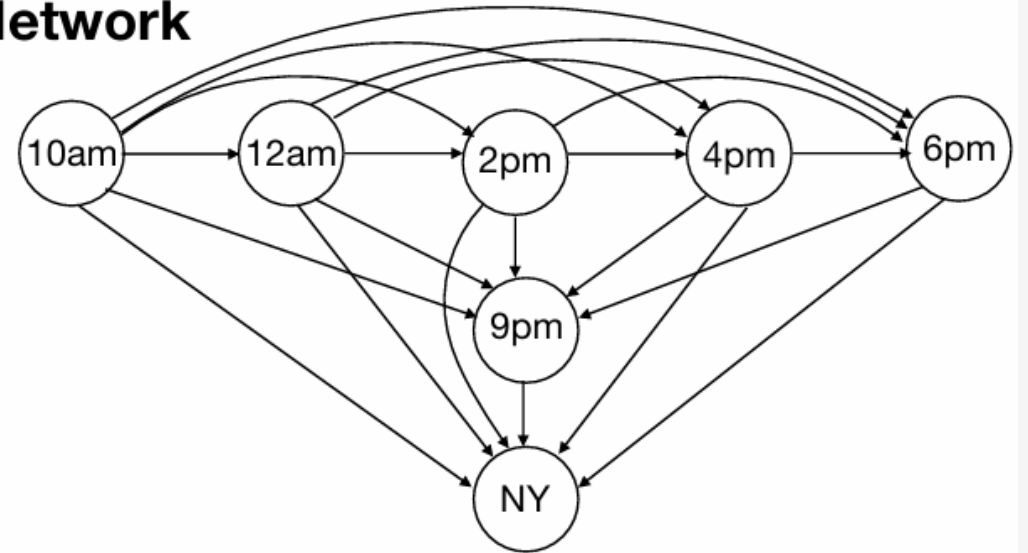
Network



Example – Airline passenger routing

- **Decision variables:**
 - x_j : passengers “flowing” on arc j
- **Costs:**
 - c_j : cost of moving a passenger on arc j
 - Between flights: \$50/hour
 - To 9pm flight: \$75 additional
 - To NY: \$0 (as scheduled)
- **Supplies:**
 - b_i : reserved passengers for flight i
 - 9pm flight: $b_i = 0$
 - NY supply: -total reserved passengers
- **Capacities:**
 - u_j : maximum passengers over arc j
 - Between flights: $u_j = \infty$
 - To NY: u_i = flight capacity

Network



Minimum cost network flow:

$$\min c^T x$$

$$s.t. Ax = b$$

$$0 \leq x \leq u$$